

CS219

CLOUD COMPUTING

WEEK 07

Specialized Cloud Mechanism

These slides are based on

Chapter 8

Specialized Cloud Mechanisms

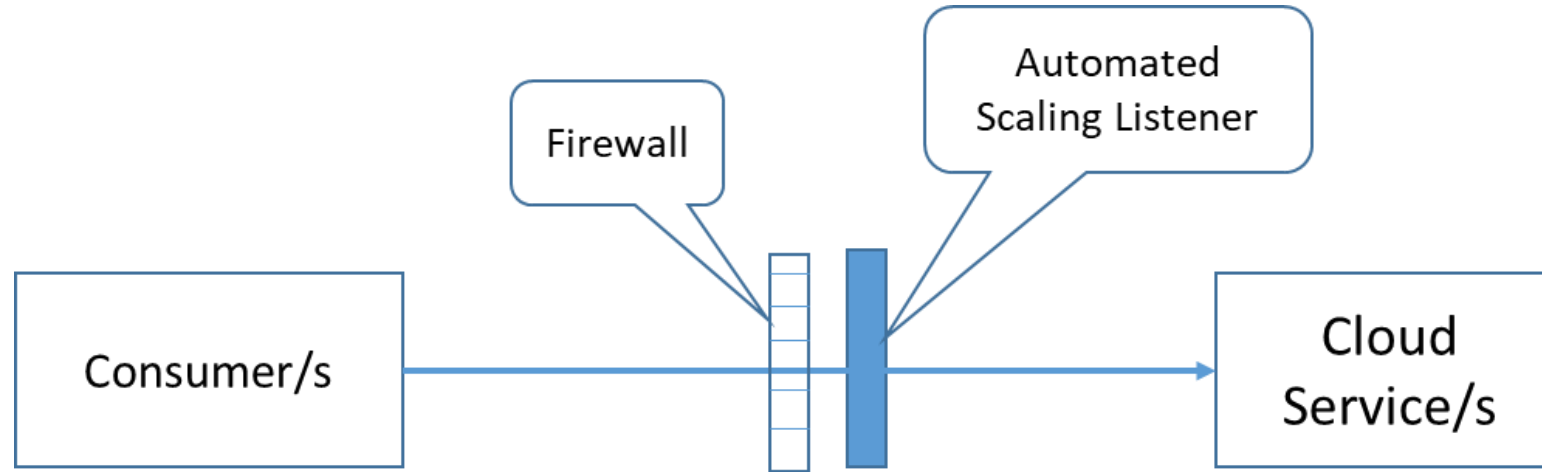
Book

Erl, Thomas, Ricardo Puttini, and Zaigham Mahmood. *Cloud computing: concepts, technology, & architecture*. Pearson Education, 2013.

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Automated Scaling Listener



- It is a *service agent* that *monitors and keeps track* of communication between service consumers and the services *for load balancing purposes*.
- It resides within the cloud, typically *near the firewall*, from where it *automatically tracks load status* information.

Automated Scaling Listener – Advantages

- Can provide different types of responses to *load fluctuation* conditions.
 - It can automatically *scale out* or *scale in* IT resources based on *parameters previously defined* by the cloud consumer.
 - ✓ Commonly referred to as *auto-scaling*.
 - It can *automatically notify the cloud consumer* when loads exceed current thresholds or are falling below allocated resources (*Figure 8.1*, in next slide).
 - ✓ This way, the cloud consumer can choose to adjust its current *IT resource allocation*.

Automated Scaling Listener – cont'd

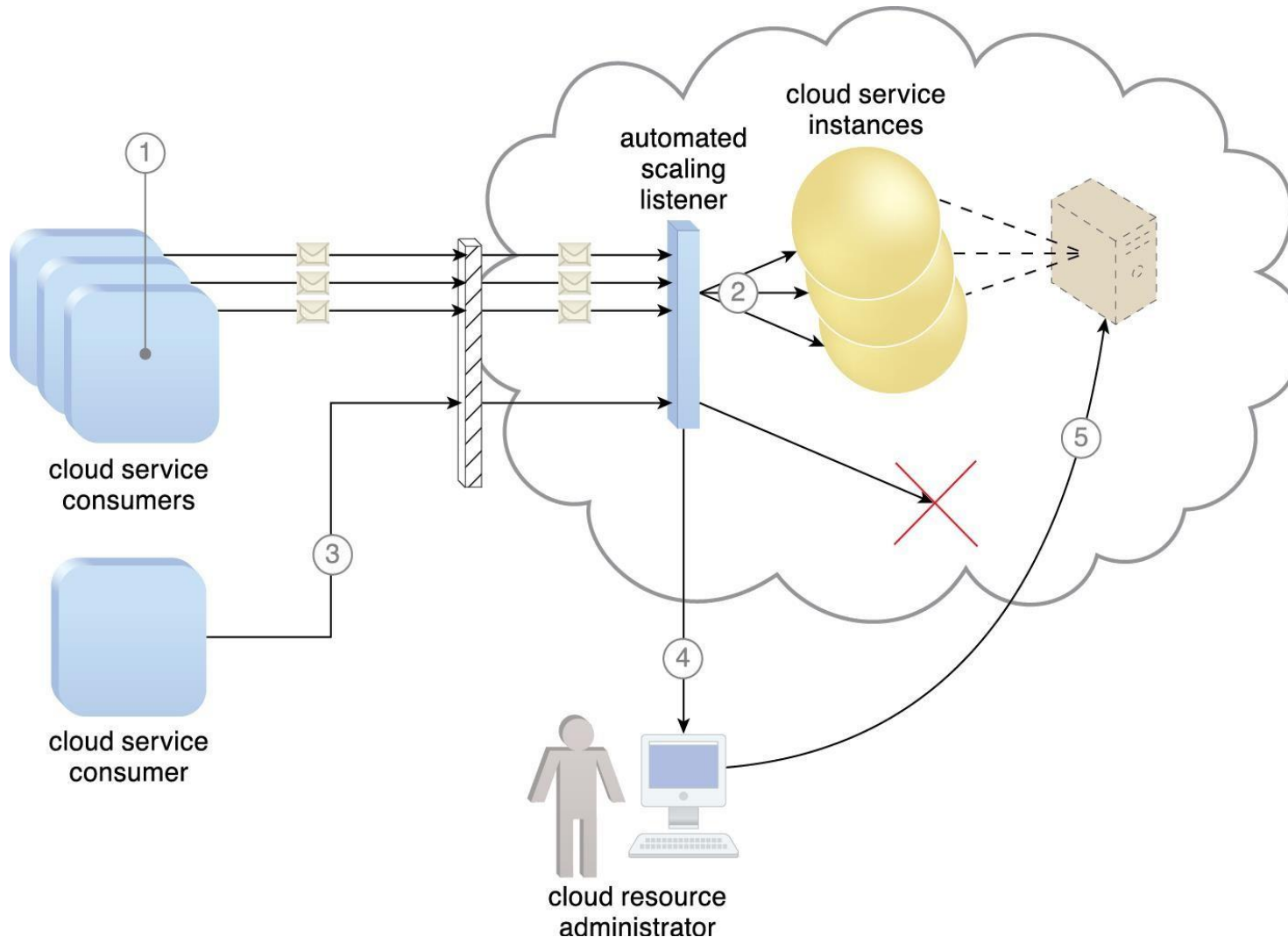


Figure 8.1

Three cloud service consumers attempt to access one cloud service simultaneously (1).

The automated scaling listener scales out and initiates the creation of three redundant instances of the service (2).

A fourth cloud service consumer attempts to use the cloud service (3).

Programmed to allow up to only three instances of the cloud service, the automated scaling listener rejects the fourth attempt and notifies the cloud consumer that the requested workload limit has been exceeded (4).

The cloud consumer's cloud resource administrator accesses the remote administration environment to adjust the provisioning setup and increase the redundant instance limit (5).

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Load Balancer

- The load balancer mechanism is *a runtime agent* with pre-programmed logics – primarily for *horizontal scaling*.
- A common purpose of *horizontal scaling* is workload balancing across two or more IT resources to *increase the performance and the capacity* that a single IT resource cannot provide.

Load Balancer – cont'd

- Load balancers can perform a range of specialized *runtime workload distribution* functions, such as:
 - **Asymmetric Distribution** - *Issuing larger* workloads to IT resources with *higher processing capacities*.
 - **Workload Prioritization** - *Scheduling, queuing, discarding, and distributing* workloads according to their *priority levels*.
 - **Content-Aware Distribution** - Distributing requests to different IT resources as *dictated by the content* of each request.

Load Balancer – cont'd

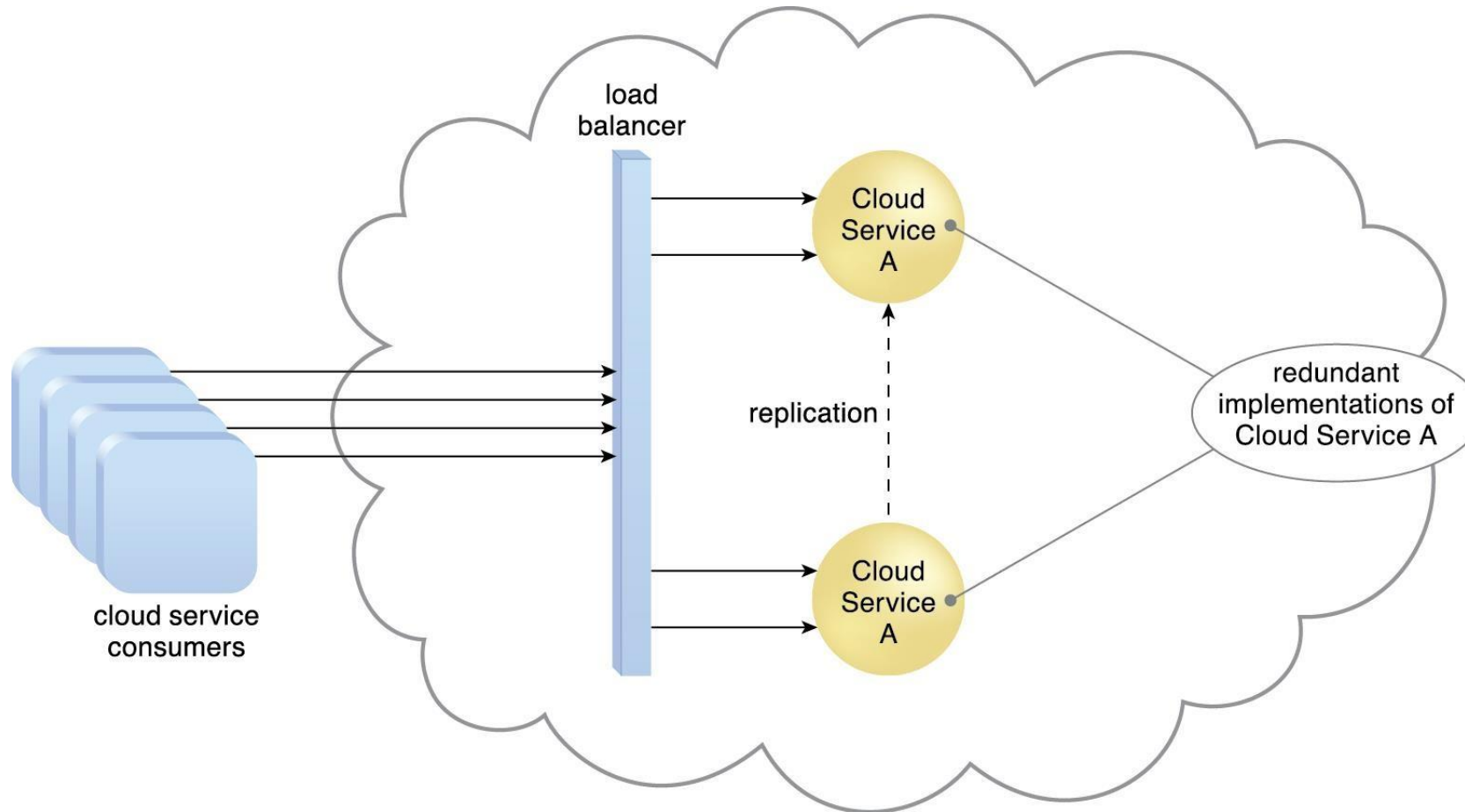


Figure 8.5

A load balancer implemented as a service agent transparently distributes incoming workload request messages across *two redundant cloud service implementations*, which in turn maximizes performance for the cloud service consumers.

Different form of Load Balancer

- Load balancer mechanisms *consists* of:
 - Multi-Layer Network Switch
 - Dedicated Load Balancer Hardware Appliance
 - Dedicated Software-Based
 - Service Agent

Load Balancer as a Service

- Enables networking to *distribute incoming requests evenly* among designated instances.
 - Distribution ensures that the workload is shared *predictably* among instances and *enables* more *effective use* of system resources.
- Load balancing methods to distribute incoming requests:
 - **Round Robin:** *Rotates requests evenly* between multiple instances.
 - **Source IP:** *Requests from a unique source IP* address are consistently *directed* to the same instance
 - **Least Connection:** Allocates requests to the instance with the *least number of active connections*.

Load Balancer Scheduling (Round-Robin)

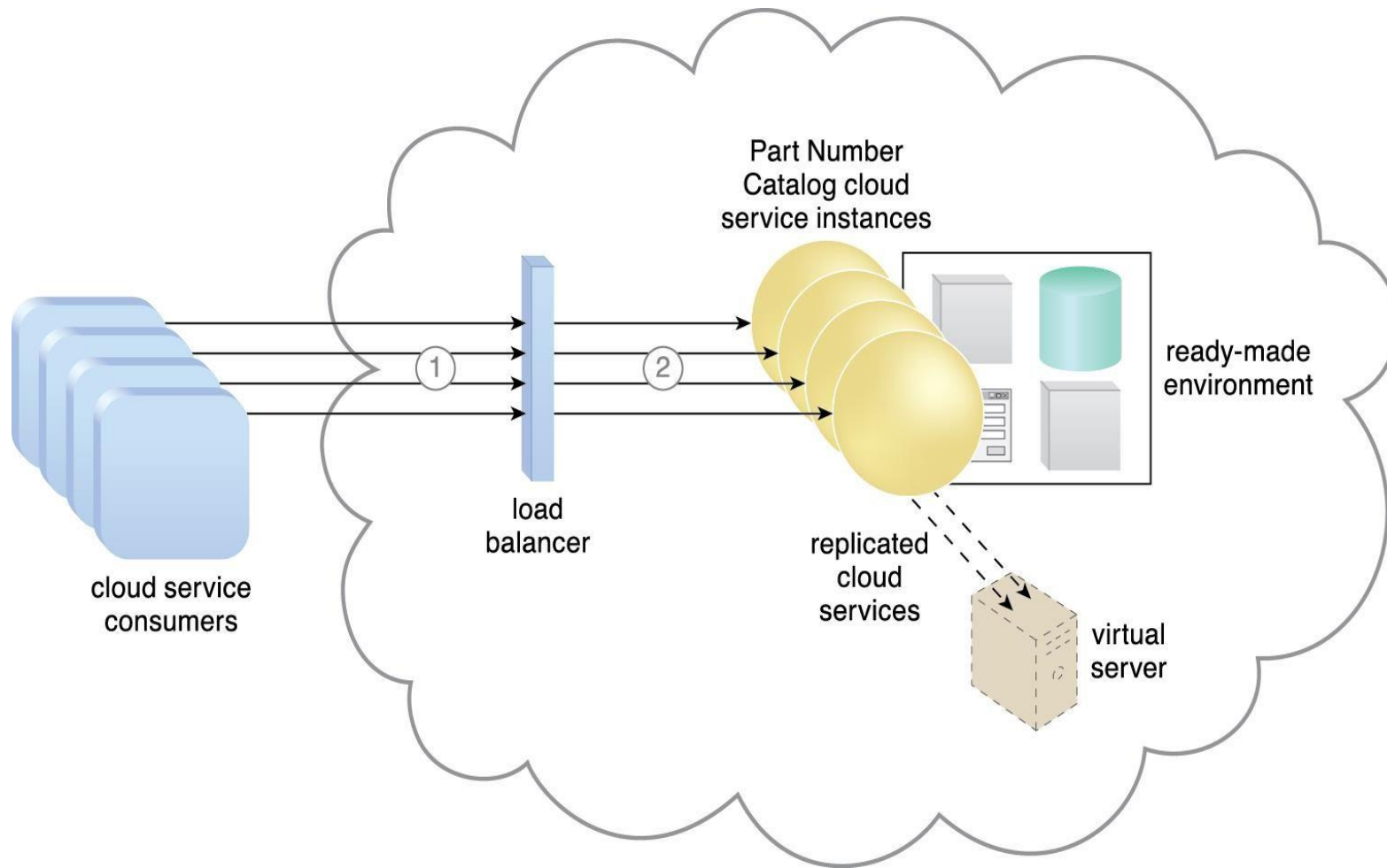


Figure 8.6

New instances of the cloud services are automatically created to meet increasing usage requests. The load balancer uses *round-robin scheduling* to ensure that the traffic is distributed evenly among the active cloud services.

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

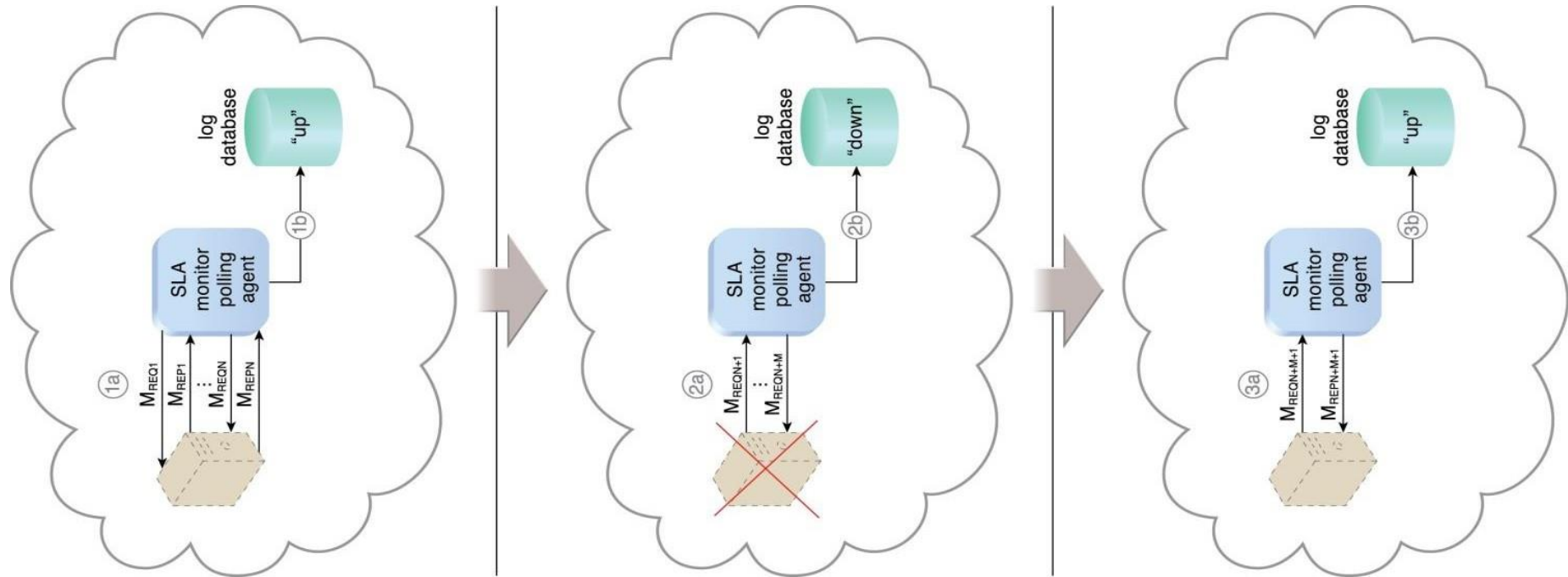
Service Level Agreement (SLA) Monitor

- The SLA monitor mechanism is specifically used to *observe the runtime performance* of cloud services to ensure that they are *fulfilling* contractual *quality-of-service (QoS)* requirements, as published in the service level agreement.
 - The data collected by the SLA monitor is processed by an *SLA management system* to be aggregated into *SLA reporting metrics*.
 - The system can also proactively *repair or failover cloud services* when *exception* conditions *occur*, such as when the SLA monitor reports a cloud service as "*down*".

SLA Monitor as a Polling Agent

Figure 8.7

The SLA monitor polls the cloud service by sending over polling request messages (M_{REQ1} to M_{REQN}). The monitor receives polling response messages (M_{REP1} ... M_{REPN}) that report that the service was “up” at each polling cycle (1a). The SLA monitor stores the “up” time—time period of all polling cycles 1 to N—in the log database (1b).



The SLA monitor polls the cloud service that sends polling request messages (M_{REQN+1} ... M_{REQN+M}). Polling response messages are not received (2a). The response messages continue to time out, so the SLA monitor stores the “down” time—time period of all polling cycles N+1 to N+M—in the log database (2b).

The SLA monitor sends a polling request message ($M_{REQN+M+1}$) and receives the polling response message ($M_{REP N+M+1}$) (3a). The SLA monitor stores the “up” time in the log database (3b).

SLA Monitor as a Polling Agent – cont'd

- The steps in Figure 8.7 are explained as follows:
 - **Step 1 “up-time”:** The SLA monitor polls the cloud service sending polling request messages M_{REQ1} to M_{REQN} ; and receives the respective polling response messages M_{REP1} to M_{REPN} reporting that the service is 'up' at each polling cycle (1a).
 - ✓ The SLA monitor stores the 'up' time period (i.e. period of time comprised in the polling cycles 1 to N) in the log database (1b).
 - **Step 2 “down-time”:** The SLA monitor polls the cloud service sending polling request messages M_{REQN+1} to M_{REQN+M} and polling response messages are not received (2a).
 - ✓ While the response messages keeps timing out, the SLA monitor stores the 'down' time period (i.e. period of time comprised in the polling cycles N+1 to N+M) in the log database (2b).
 - **Step 3 “up-time”:** The SLA monitor sends polling request message $M_{REQN+M+1}$; and receives the polling response message $M_{REPN+M+1}$ (3a).
 - ✓ The SLA monitor stores the 'up' time period in the log database (3b).
 - The data collected by the SLA monitor is usually **processed** by a **SLA management system** that is responsible for processing the data collected during usage and aggregate it in more **manageable and meaningful SLA metrics**.

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Pay Per Use Monitor

- The monitor is implemented as a *resource agent*.
- The pay-per-use monitor mechanism *measures* cloud-based IT resource *usage based on predefined parameters* in pricing plans and *generates usage logs* for fee calculations.
- Typical monitoring *variables* are:
 - request/response message quantities
 - transmitted data volume
 - bandwidth consumption
- The *data* collected by the pay-for-use monitor is *processed by a billing system* that calculates payment fees.
- *Figure 8.12* gives an example of pay-per use

Pay Per Use Monitor

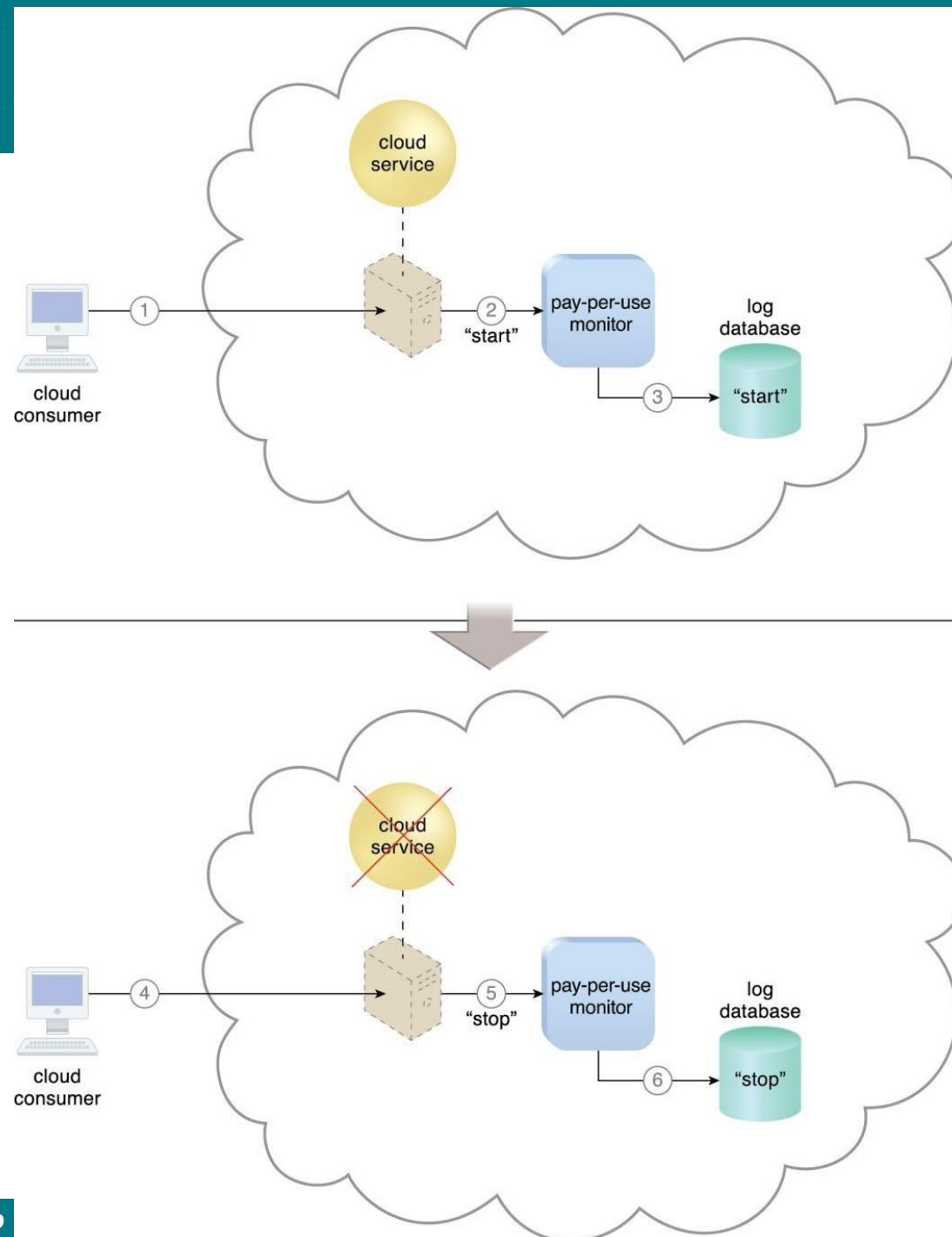


Figure 8.12

A cloud consumer requests the creation of a new instance of a cloud service (1). The IT resource is instantiated and the pay-per-use monitor receives a "start" event notification from the resource software (2). The pay-per-use monitor stores the value timestamp in the log database (3).

The cloud consumer later requests that the cloud service to instance be stopped (4). The pay-per-use monitor receives a "stop" event notification from the resource software (5) and stores the value timestamp in the log database (6).

Pay Per Use Monitor – cont'd

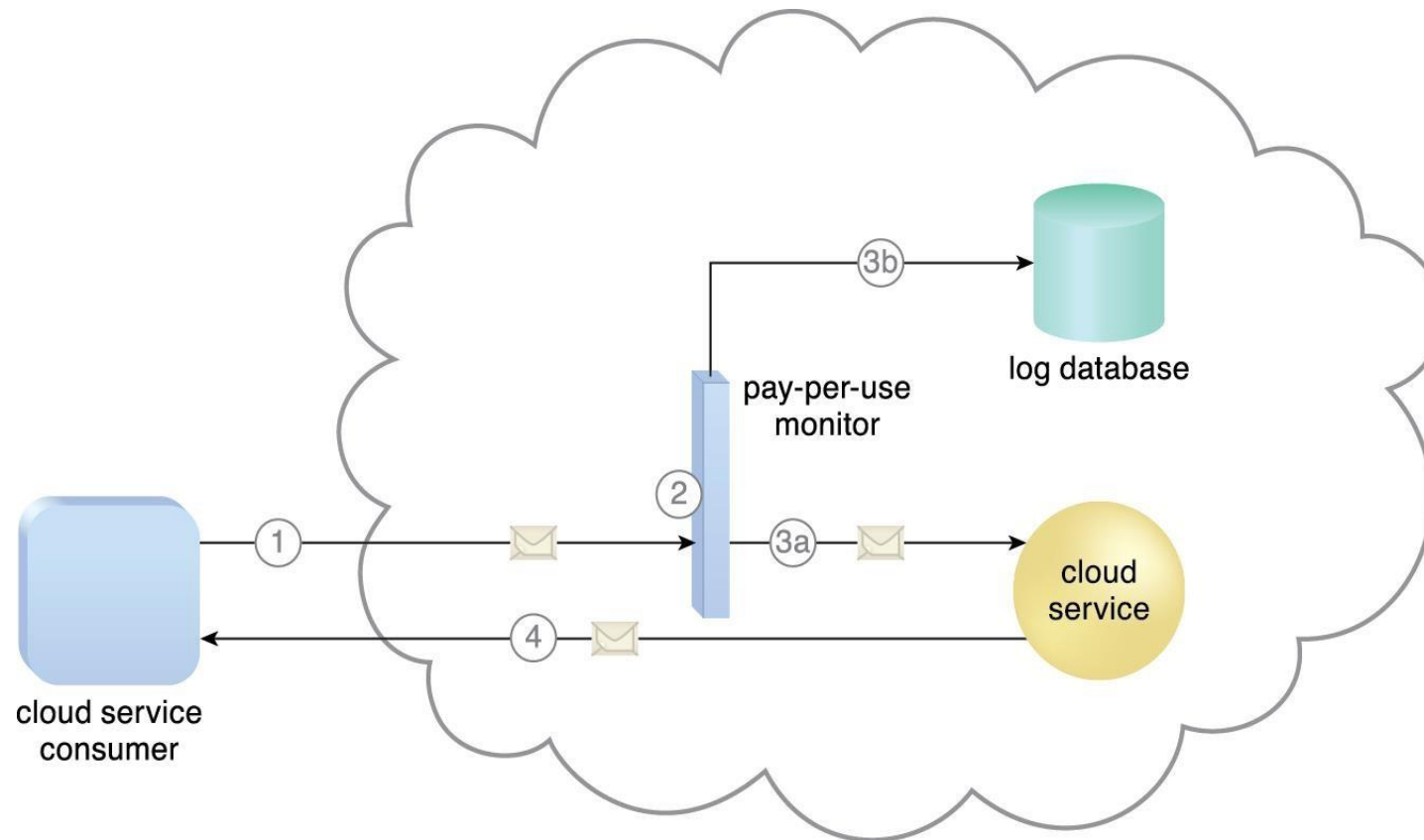


Figure 8.13 A cloud service consumer sends a request message to the cloud service (1). The pay-per-use monitor intercepts the message (2), forwards it to the cloud service (3a), and stores the usage information in accordance with its monitoring metrics (3b). The cloud service forwards the response messages back to the cloud service consumer to provide the requested service (4).

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Audit Monitor

- An audit monitor is a specialized cloud usage monitor that is used for collecting data *for network and system audits*.
- This data is commonly called *audit track* data.
 - The data collected by this mechanism is usually *specified by auditing requirements*, required by *regulatory, compliance* and consumer-provider *contract conditions*.
 - This data can be chosen from a broad selection of *monitoring data* types, depending on the auditing requirements.

Audit Monitor –cont'd

- Figure 8.15 shows an audit monitor designed as a *service agent* that intercepts '*logon*' messages and stores the cloud service consumer's security credentials and respective *login attempts* data (e.g. *timestamp*, *login successful or not*) for auditing purposes.

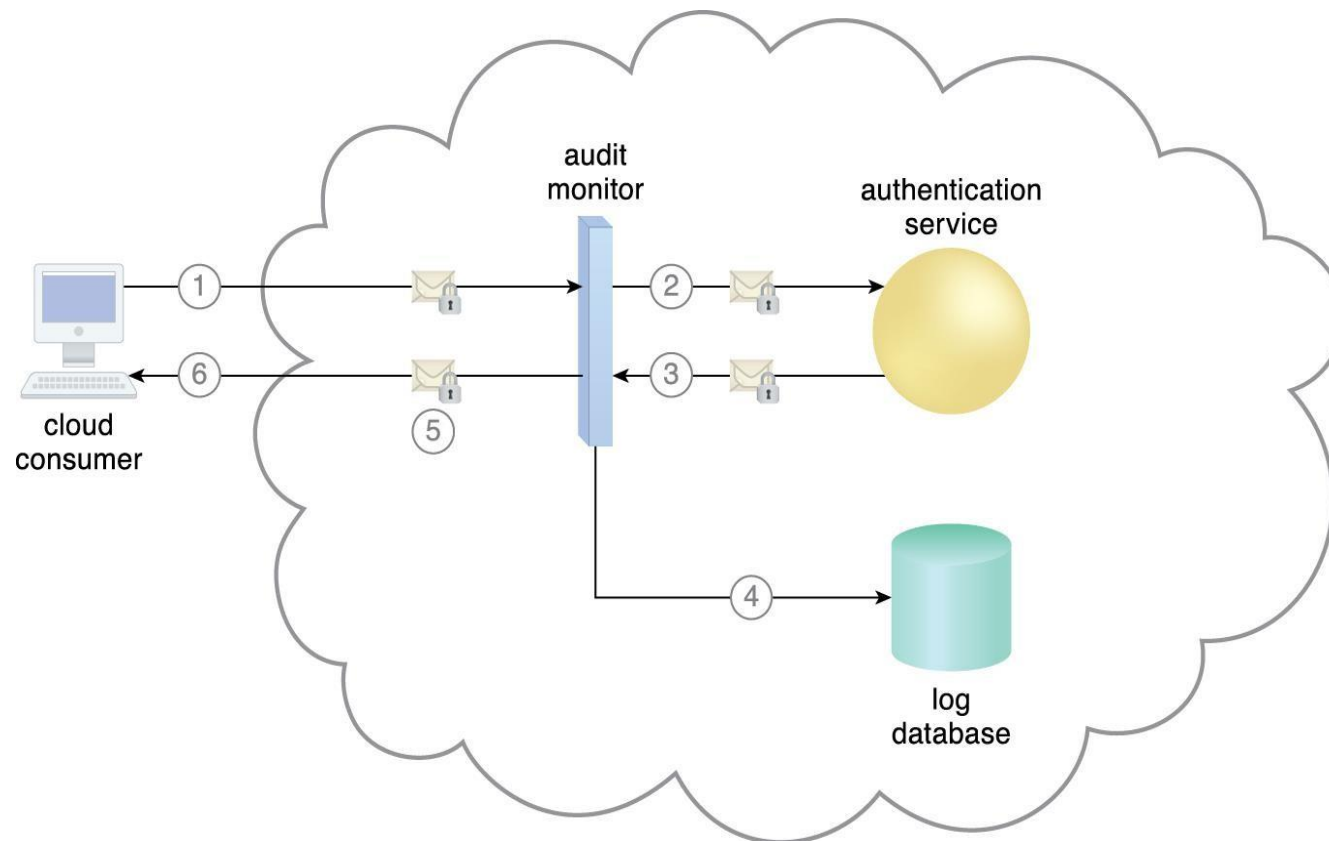


Figure 8.15

A cloud service consumer requests access to a cloud service by sending a login request message with security credentials (1). The audit monitor intercepts the message (2) and forwards it to the authentication service (3). The authentication service processes the security credentials. A response message is generated for the cloud service consumer, in addition to the results from the login attempt (4). The audit monitor intercepts the response message and stores the entire collected login event details in the log database, as per the organization's audit policy requirements (5). Access has been granted, and a response is sent back to the cloud service consumer (6).

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Failover System

- The failover system mechanism *increases reliability* and *availability* by using established *clustering technology* to provide *redundant implementations* of software programs.
 - The failover system is *configured* to *switch automatically* over to a *redundant* or *standby* IT resource instance whenever the currently *active* IT resource becomes *unavailable*.

Failover System – cont'd

- Failover systems are commonly used for *mission-critical* programs or for *reusable services* that can introduce a *single point of failure* for multiple applications.
- A failover system can *span more than one geographical region* so that each location *hosts one or more redundant* implementations of the same IT resource.
- This mechanism *rely on the resource replication mechanism* to supply the redundant IT resource instances, that are *actively monitored* to *detect errors* and unavailability conditions.
- *Figures 8.17—8. 19* demonstrates a failover system.

Failover System – cont'd

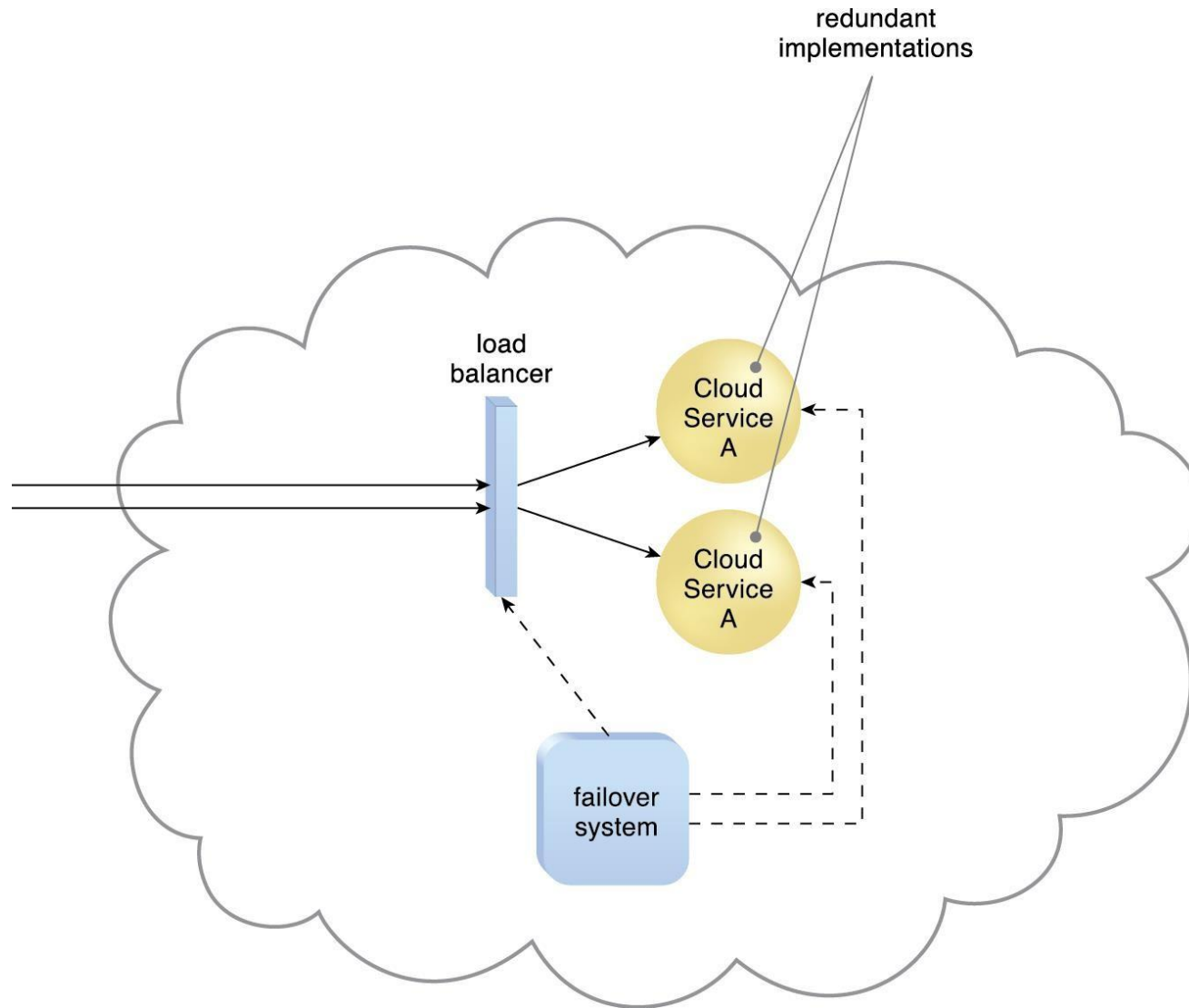


Figure 8.17

The failover system monitors the operational status of Cloud Service A.

Failover System – cont'd

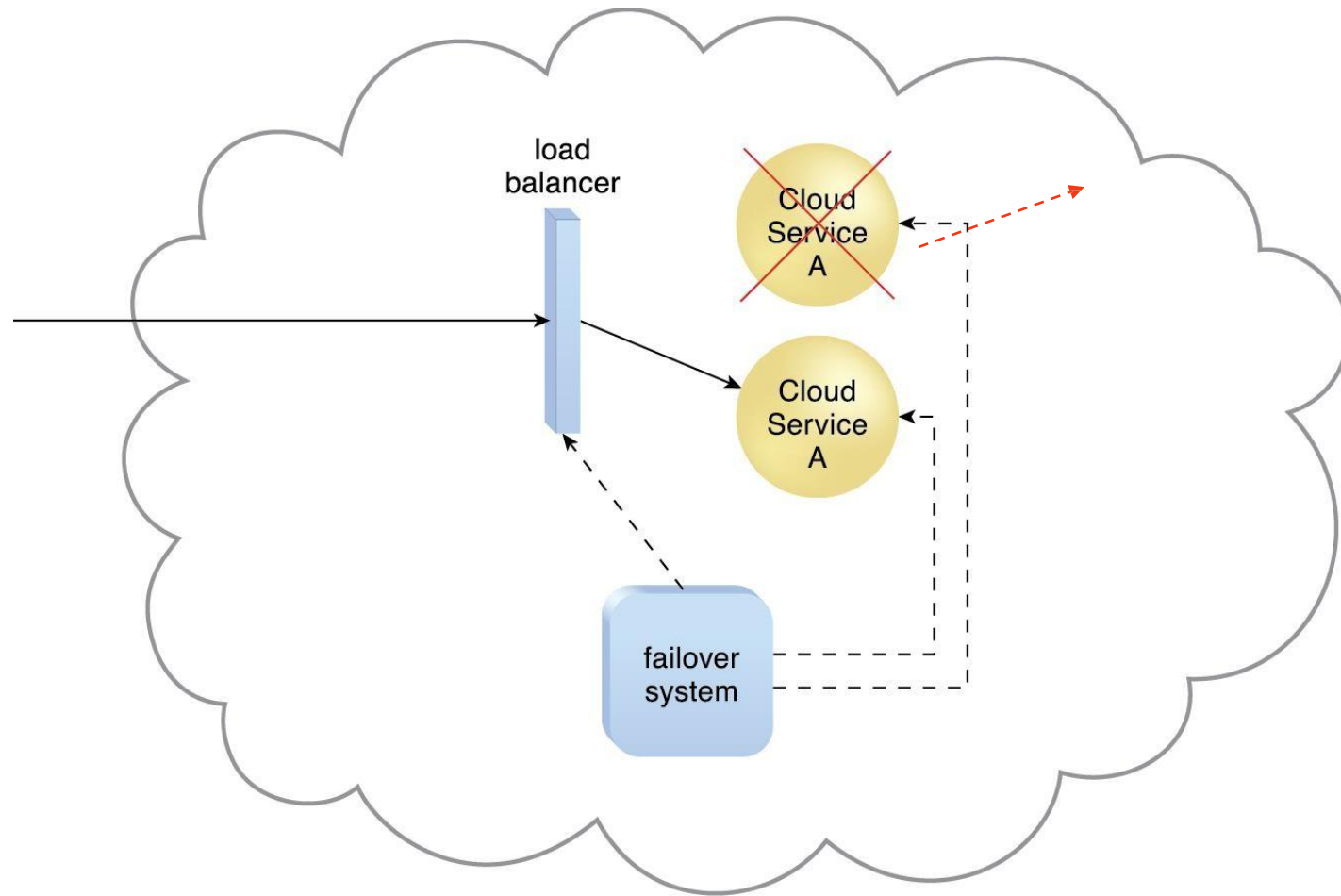


Figure 8.18

When a failure is detected in one Cloud **Service A** implementation, the failover system commands the load balancer to switch over the workload to the redundant Cloud **Service A** implementation.

Failover System – cont'd

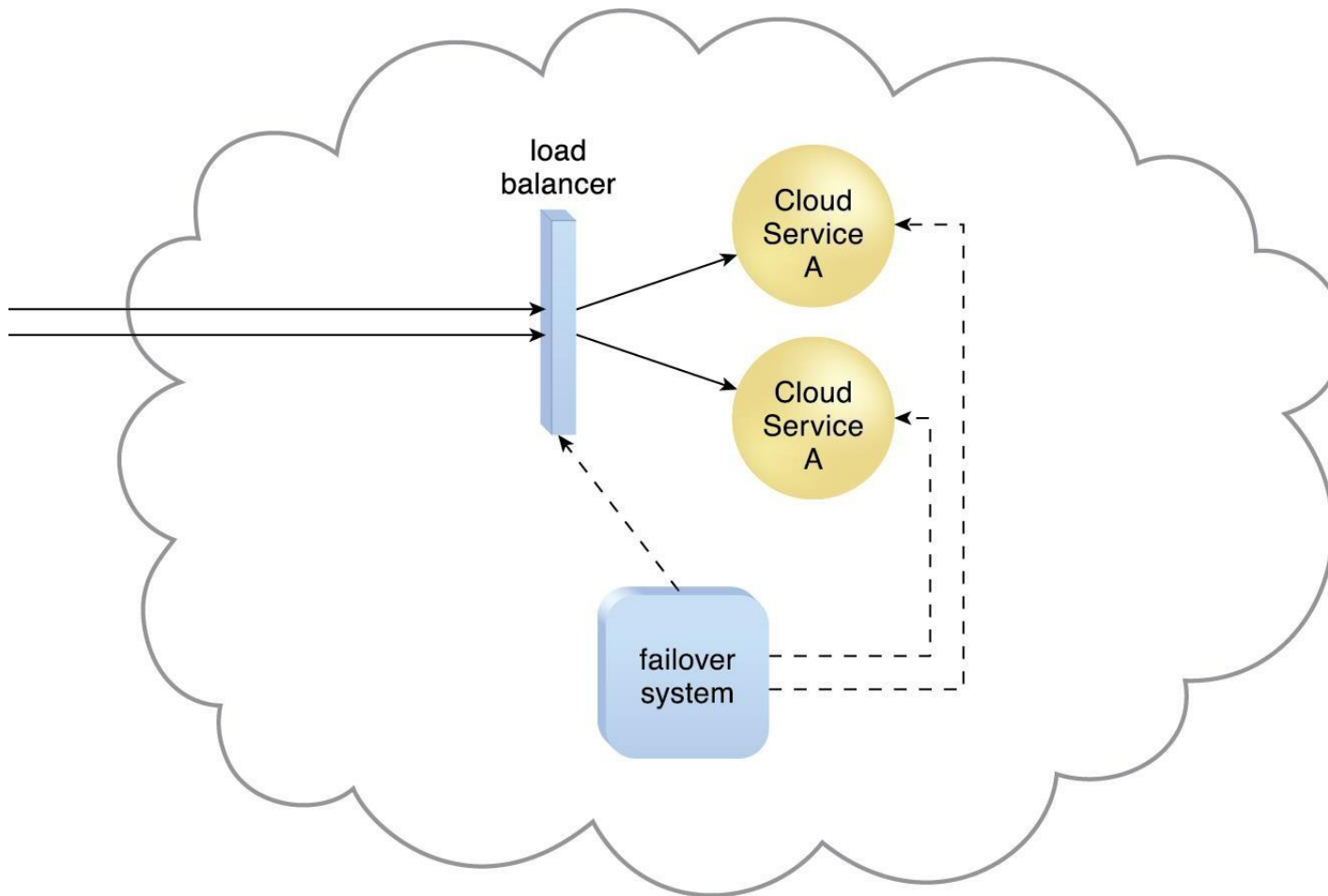


Figure 8.19

The failed Cloud **Service A** implementation is recovered or replicated into an operational cloud service. The failover system now commands the load balancer to distribute the workload again.

Failover System – cont'd

- Failover systems come in two basic configurations:
 - Active-Active
 - Active-Passive

Active-Active Failover System

- Redundant implementations of the IT resource *actively serves* the workload *synchronously*.
 - An operational implementation takes over the processing whenever an IT resource failure is detected.
 - An example configuration is given in Figures 8.20 and 8.21.

Active-Active Failover System – cont'd

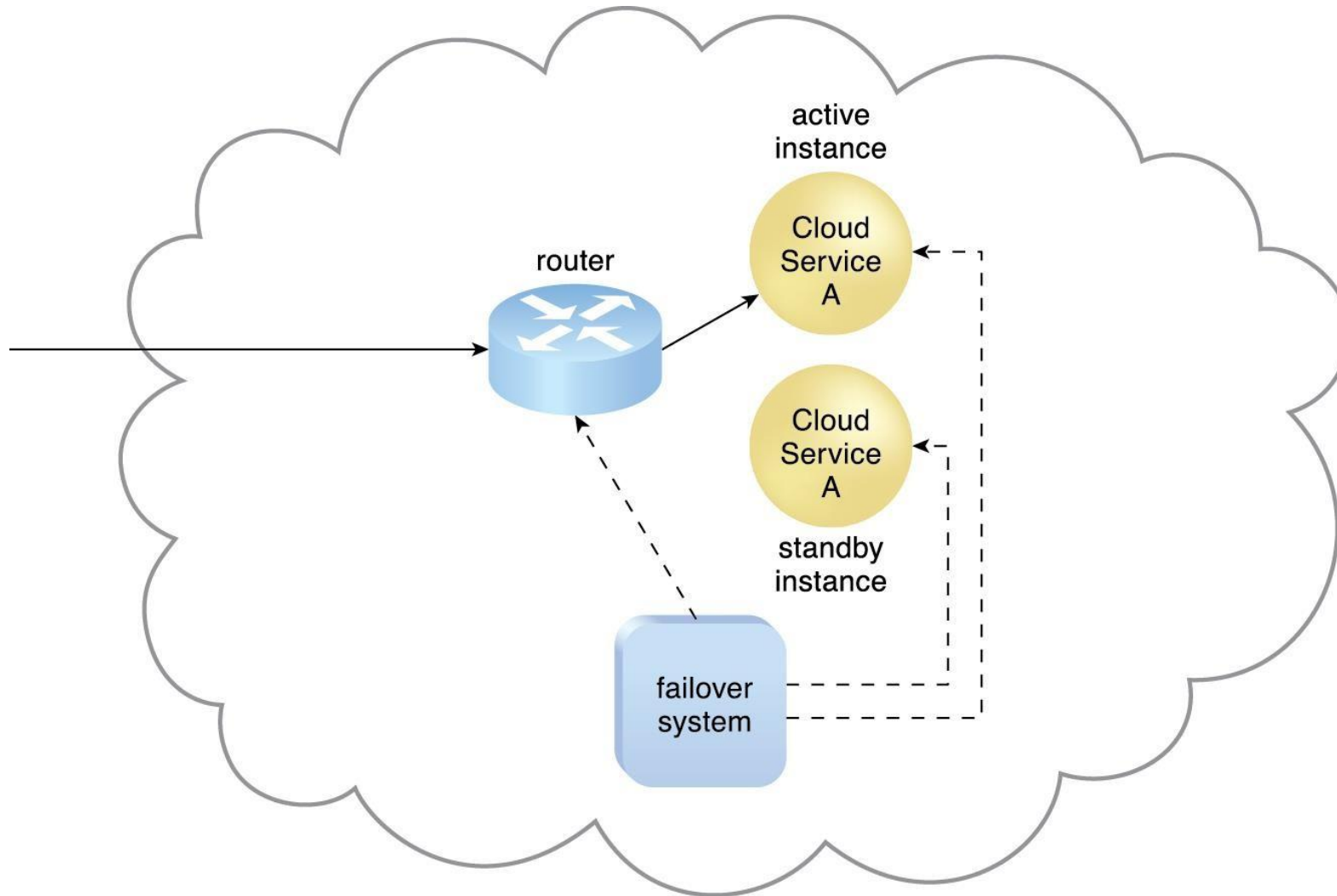


Figure 8.20

The failover system monitors the operational status of Cloud *Service A*. The Cloud *Service A* implementation acting as the active instance is receiving cloud service consumer requests.

Active-Active Failover System – cont'd

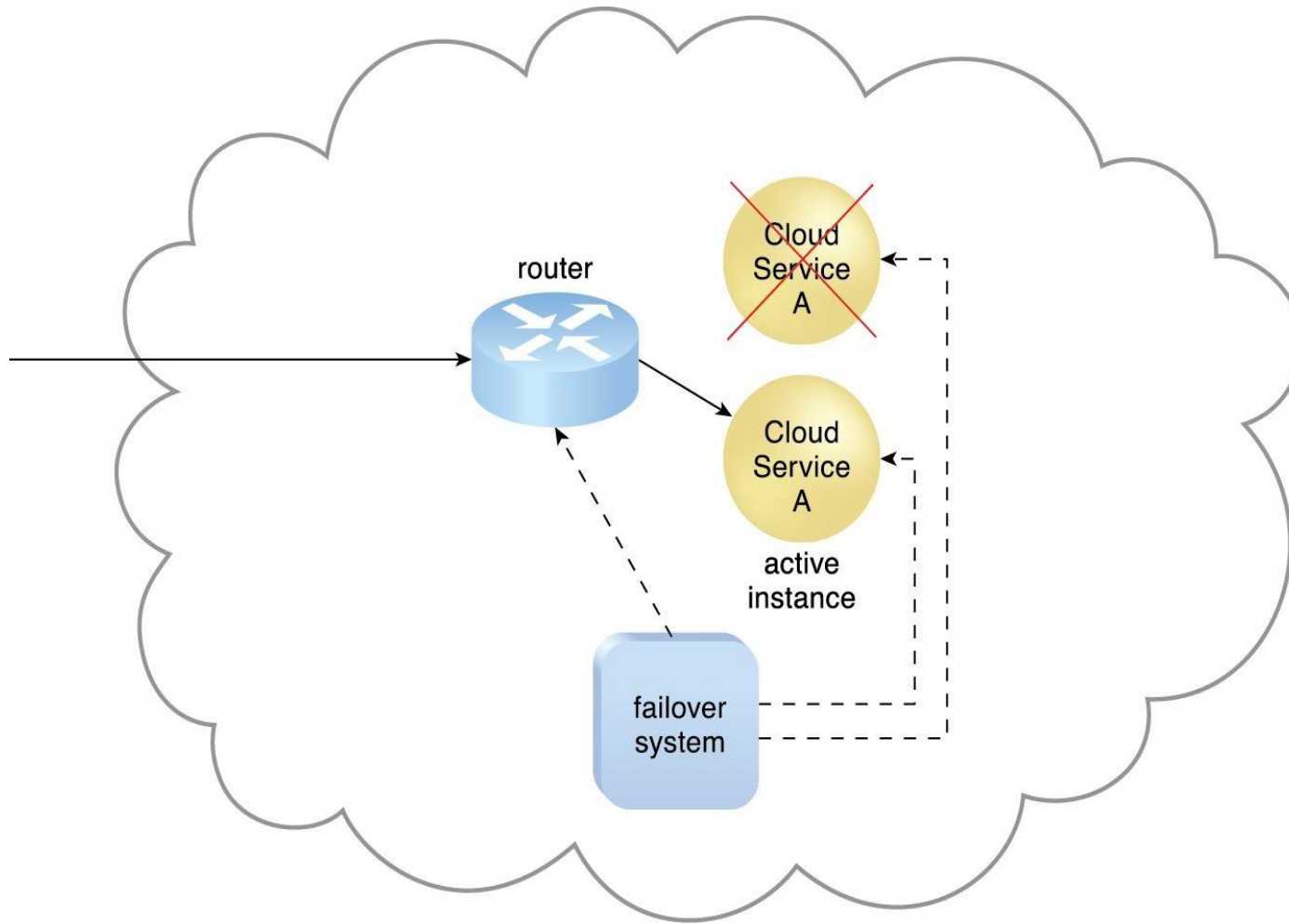


Figure 8.21

The Cloud *Service A* implementation acting as the active instance encounters a failure that is detected by the failover system, which subsequently activates the inactive Cloud *Service A* implementation and redirects the workload toward it. The newly invoked Cloud *Service A* implementation now assumes the role of active instance.

Active-Passive Failover System

- A standby or inactive implementation is activated to take over the processing from the IT resource that became unavailable.
 - Example standby configuration is shown in Figure 8.22.

Active-Passive Failover System – cont'd

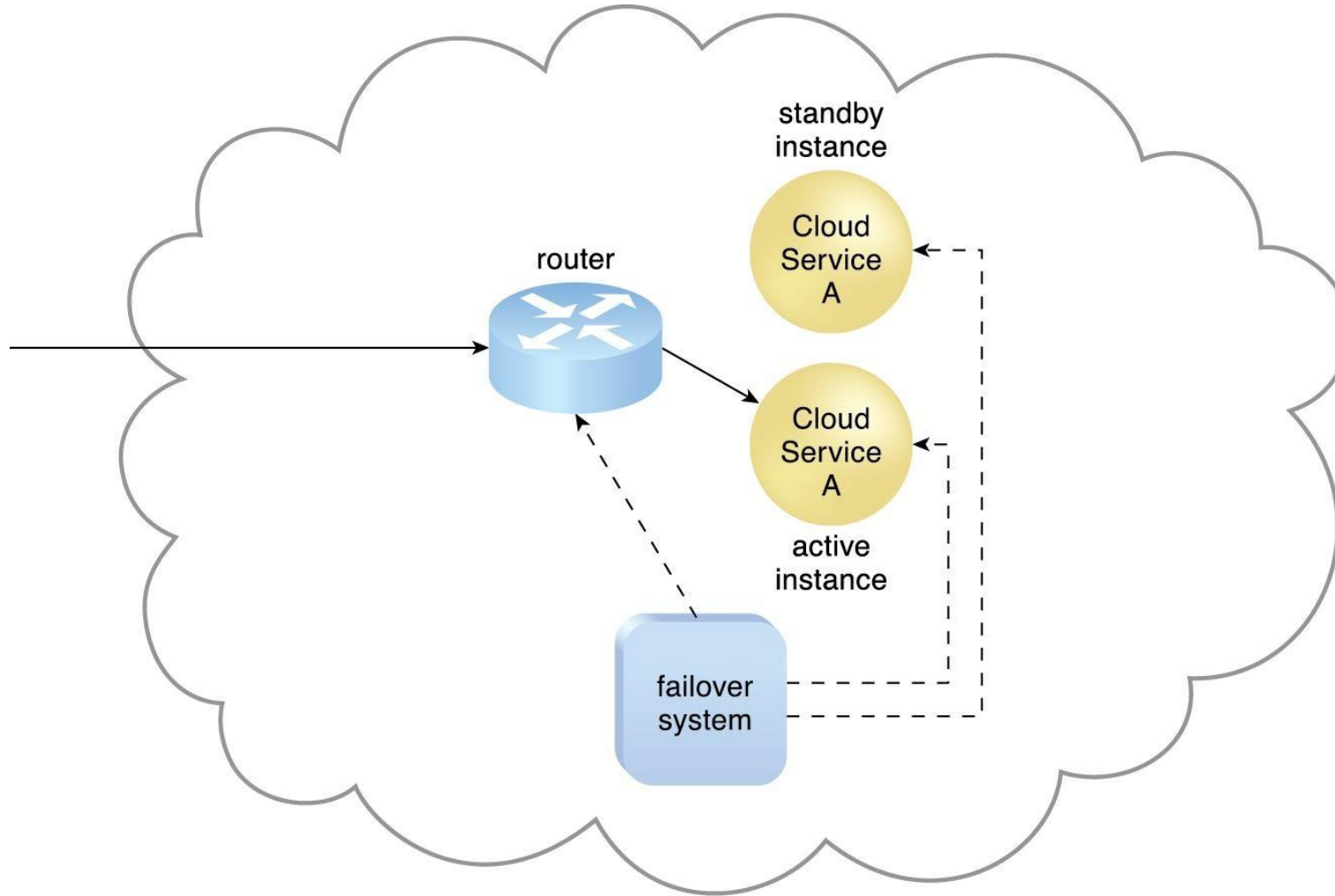


Figure 8.22

The failed Cloud Service A is *recovered* or *replicated* an operational cloud service, and is now *positioned* as the *standby* instance, while the previously invoked Cloud Service A continues to serve as the active instance.

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Hypervisor

- This mechanism is a *fundamental* part of *virtualization* infrastructure that is primarily used to generate *virtual server instances* of a physical server.
- A hypervisor is generally limited to one physical server and can therefore *only create virtual images of underlying server* (Figure 8.27).
- Similarly, a hypervisor can *only assign* virtual servers it generates to resource pools *that reside* on the *same underlying physical server*.
- A hypervisor has *limited virtual server management features*, such as *increasing* the virtual server's capacity or *shutting it down*.
- The *VIM* provides a range of features for administering multiple hypervisors across physical servers.

Virtualization using Hypervisor

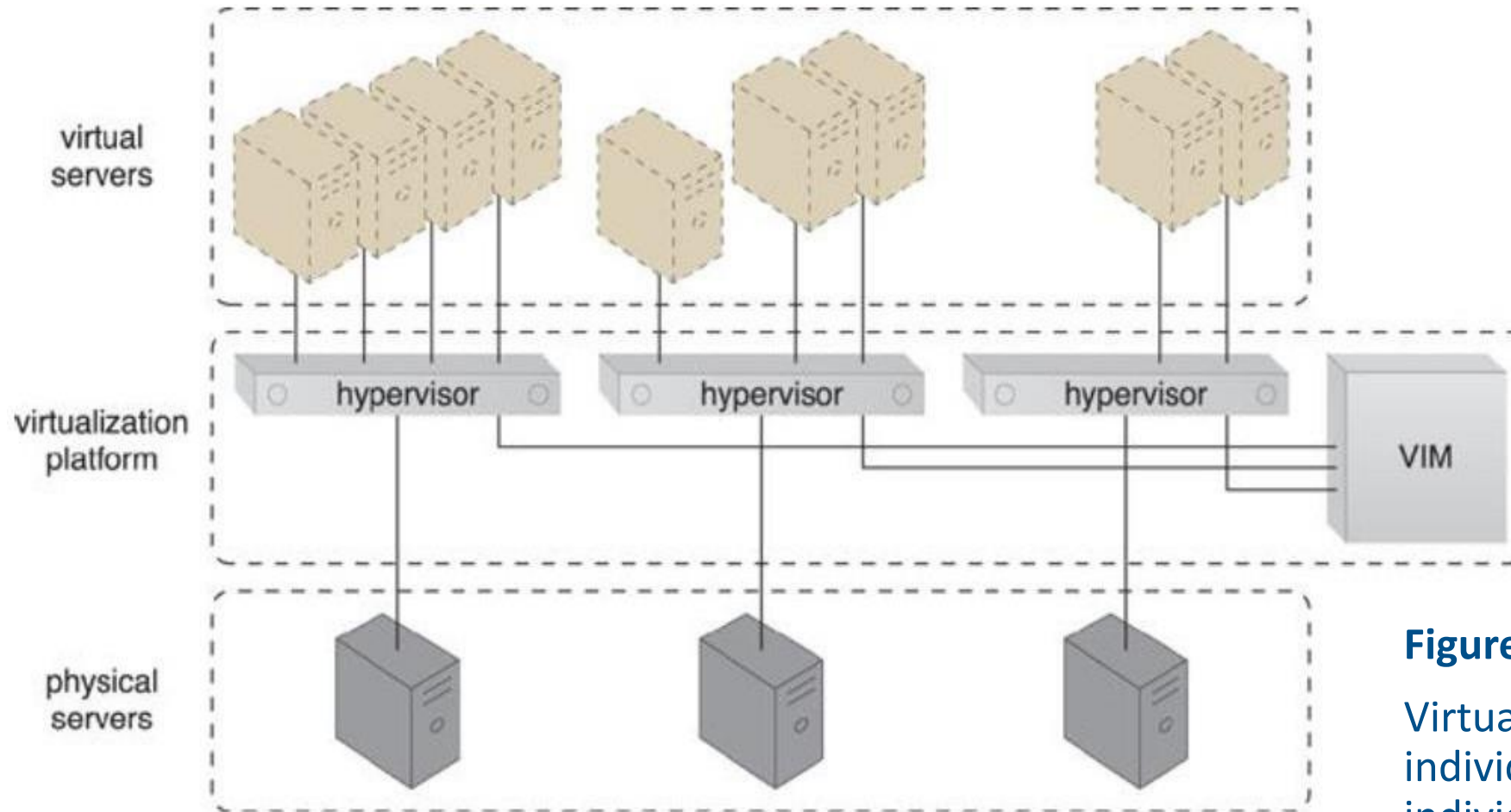


Figure 8.27.

Virtual servers are created via individual hypervisor on individual physical servers. *All three hypervisors are jointly controlled by the same VIM.*

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Resource Cluster

- Cloud-based IT resources that are *geographically diverse*, can be logically combined into groups to improve their allocation and use.
 - The resource *cluster mechanism* is used to group multiple IT resource instances so that they can be *operated as a single IT resource*. Example is shown in *Figure 8.30*.
 - This *increases* the combined *computing capacity, load balancing, and availability* of the clustered IT resources.

Resource Cluster – cont'd

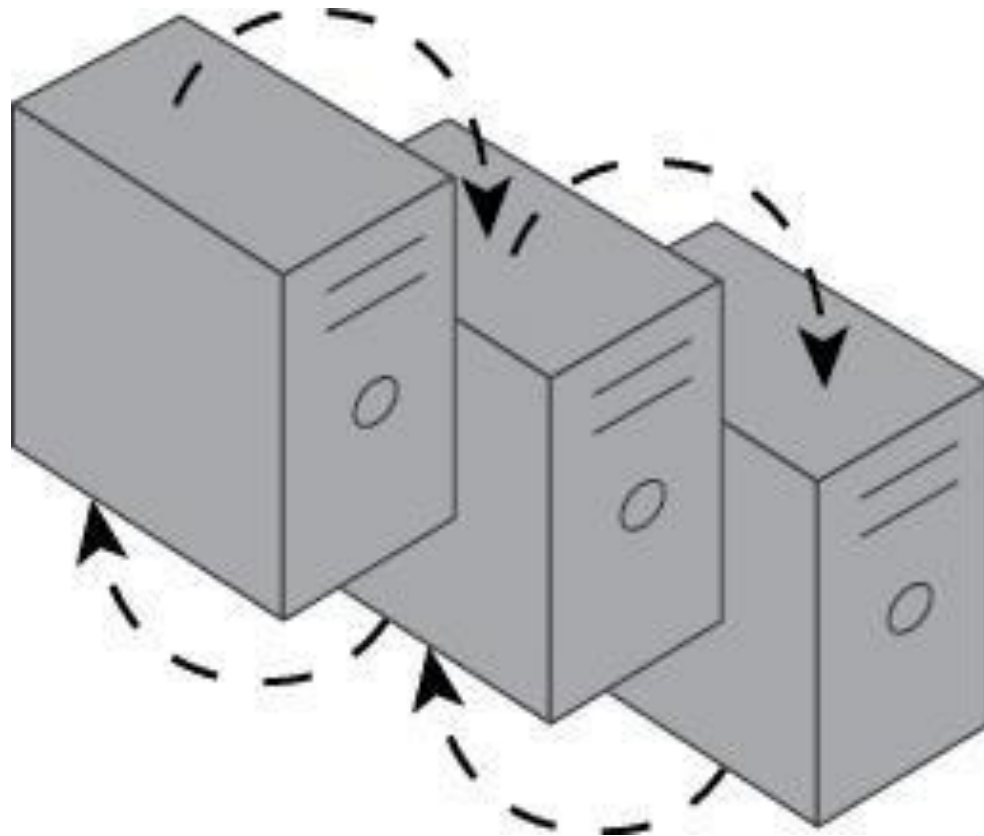


Figure 8.30 The curved dashed lines are used to indicate that IT resources are clustered.

Resource Cluster – cont'd

- Resource cluster architectures rely on *high-speed dedicated network connections* between IT resource instances (nodes) to communicate for *workload distribution, task scheduling, data sharing*, and *system synchronization*.
 - A *cluster management platform* that is running as distributed middleware in all of the cluster nodes is usually *responsible for these activities*.
 - This platform *implements a coordination function* that allows distributed IT resources to *appear as one IT resource*, and also *executes IT resources inside the cluster*.

Resource Cluster – cont'd

- Following are the common resource cluster types.
 - Server Cluster
 - Database Cluster
 - Large Dataset Cluster

Server Cluster

- Physical and virtual servers *alike* are clustered to increase the performance and availability (within IaaS environments).
 - *Hypervisors running at different physical servers* can be configured to share the virtual server execution state (*memory pages, processor register state*) in order to establish clustered virtual servers.
- In a virtual cluster environment, virtual servers are able to *live-migrate* from one physical server to another.
 - In this process, the virtualization platform *suspends the execution* of a given virtual server *at one* physical server and *resumes it* on *another* physical server

A Server Cluster

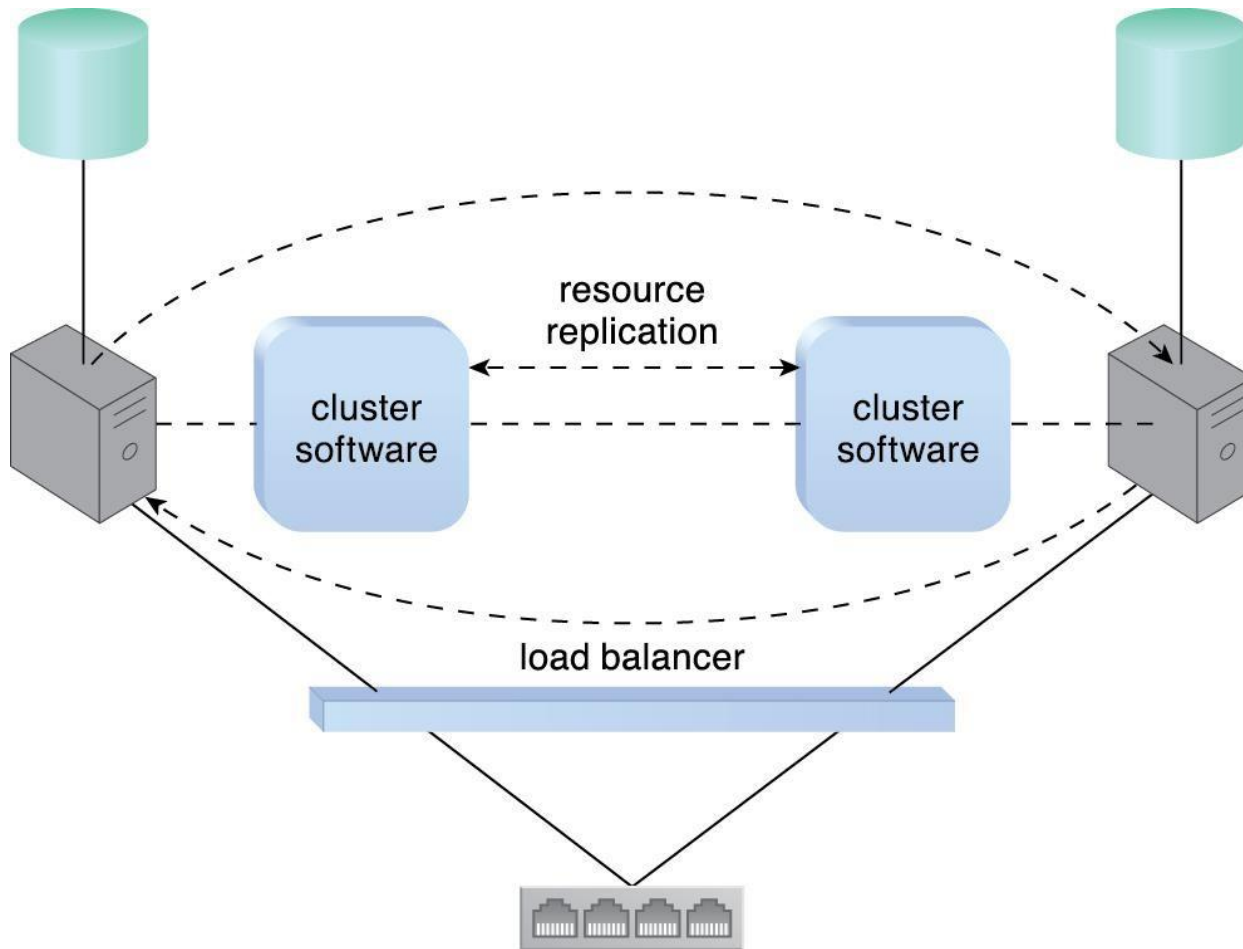


Figure 8.32

A loosely coupled server cluster that incorporates a load balancer. There is no shared storage. Resource replication is used to replicate cloud storage devices through the network by the cluster software.

Database Cluster

- This *high-availability* resource cluster *has a synchronization feature* that maintains the consistency of data being stored at different storage devices used in the cluster—aiming at *providing high data availability*.
- The redundant capacity is usually based on an *active-active* or *active-passive failover* system committed to maintaining the synchronization conditions

Large Dataset Cluster

- *Data partitioning* and distribution is implemented so that the target dataset can be *efficiently partitioned without compromising data integrity* or computing *accuracy*.
- Each cluster node *processes workloads without communicating with other nodes* as much as in other cluster types.

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Multi-Device Broker

- An *individual cloud service* may need to be accessed by *different types of cloud service consumers*, some of which may be *incompatible* with the cloud service's *published service* contract.
- Disparate cloud service consumers may be differentiated by their hosting hardware devices and/or may have different types of communication requirements.
 - *To overcome incompatibilities* between a cloud service and a disparate cloud service consumer, *mapping logic needs to be created* to transform (or convert) information that is exchanged at runtime.
- The *multi-device broker mechanism is used to facilitate runtime data transformation* so as to make a cloud service accessible by a wider range of cloud service consumer programs and devices (shown in *Figure 8.35*).

Multi-Device Broker – cont'd

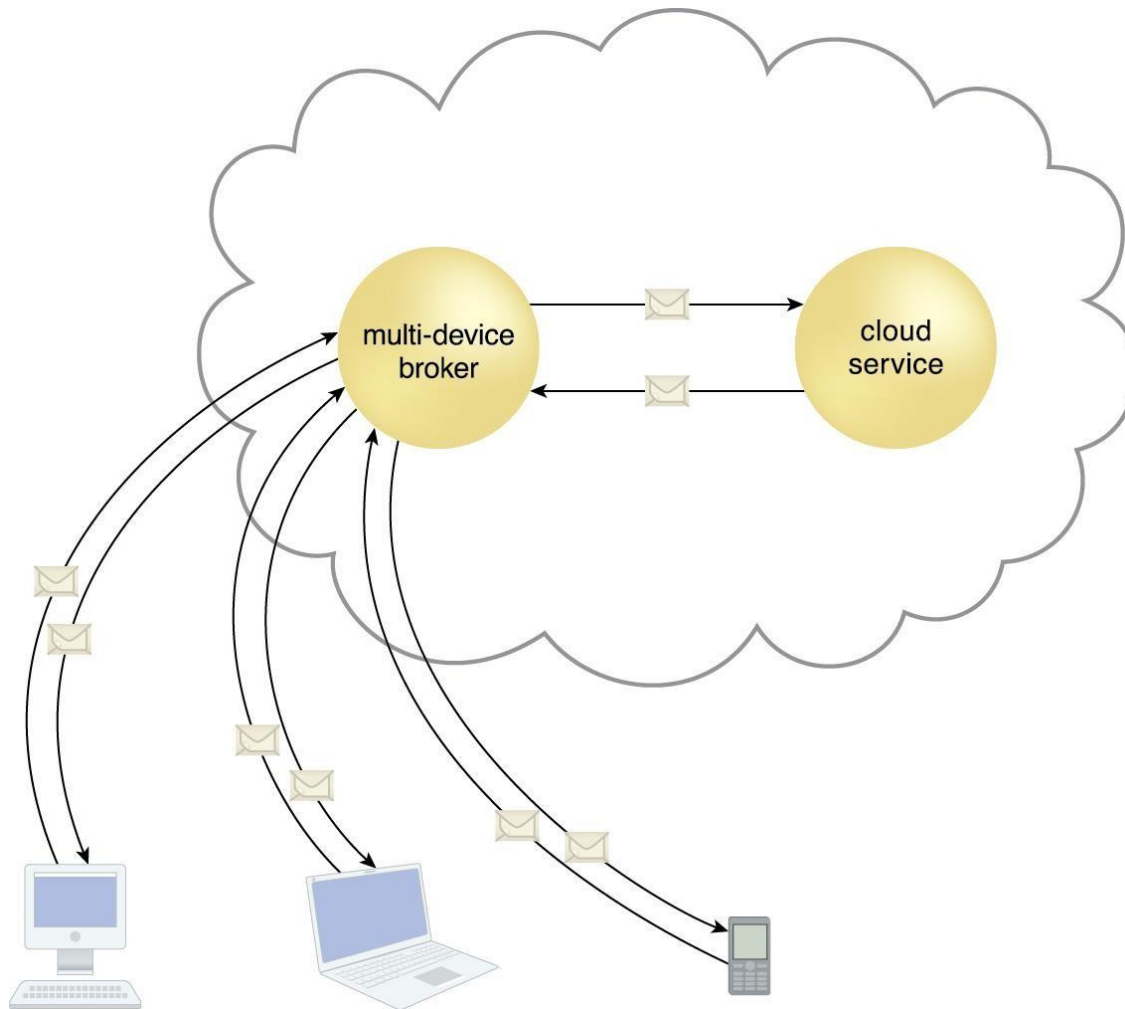


Figure 8.35

A multi-device broker contains the mapping logic necessary to transform data exchanges between a cloud service and different types of cloud service consumer devices. This scenario depicts the multi-device broker as a cloud service with its own API. This mechanism can also be implemented as a service agent that intercepts messages at runtime to perform necessary transformations.

Multi-Device Broker – cont'd

- Multi-device brokers commonly exist as or incorporate gateway components, such as:
 - **XML Gateway** - transmits and validates XML data
 - **Cloud Storage Gateway** - transforms cloud storage protocols and encodes storage devices to facilitate data transfer and storage
 - **Mobile Device Gateway** - transforms the communication protocols used by mobile devices
- The *levels* at which *transformation logic* can be created include:
 - transport protocols
 - messaging protocols
 - storage device protocols
 - data schemas/data models

Outlines

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

State Management Database

- State management database is a *storage device* used to *temporarily persist* state data for software programs.
- As an *alternative to caching* state data in memory, software programs can *off-load state data* to the database in order to *reduce* the amount of *runtime memory they consume*.
 - By doing so, the software programs and the surrounding infrastructure *become more scalable*.
 - State management databases are commonly used by cloud services, especially those involved in *long-running runtime activities*.

State Data in Memory

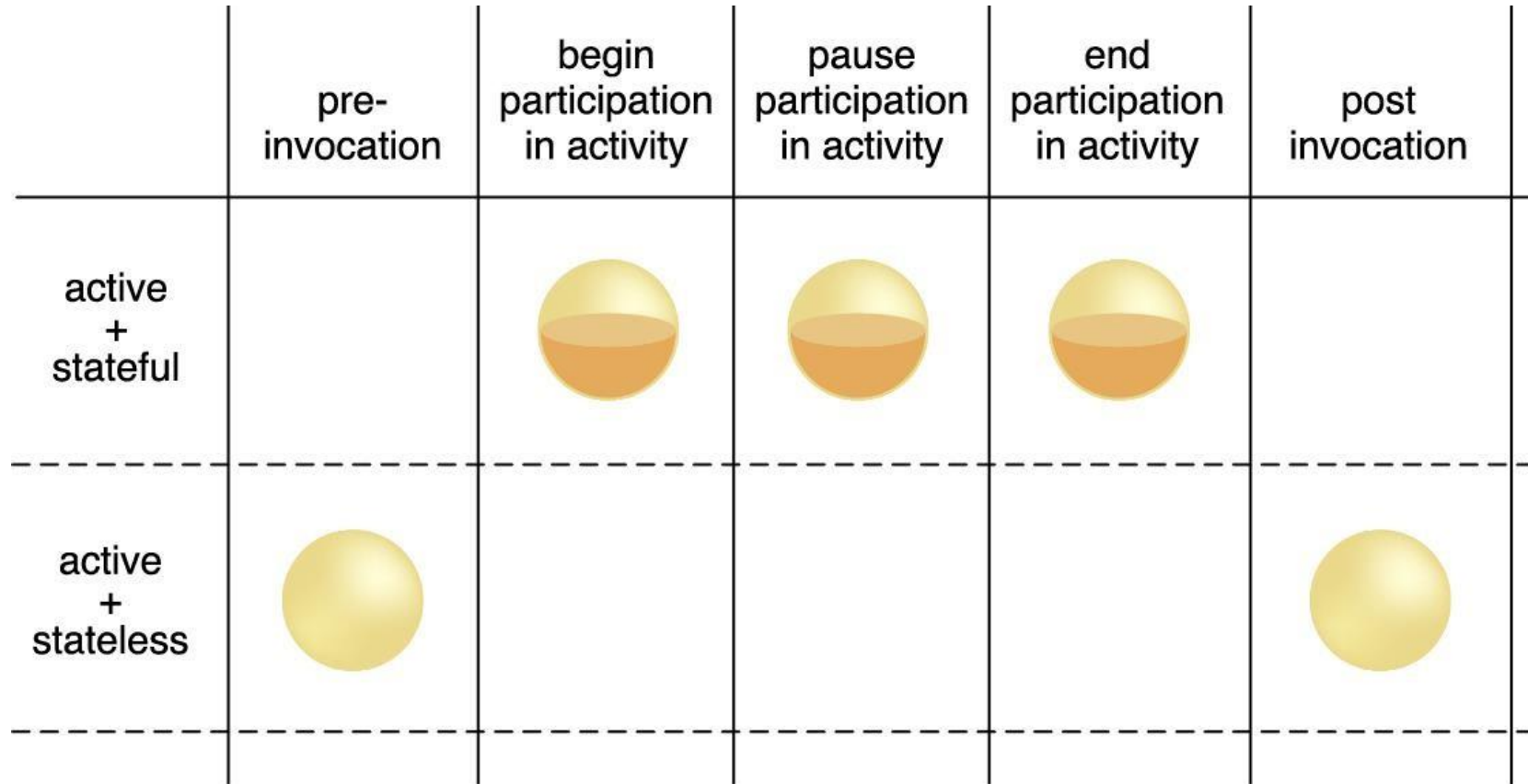


Figure 8.37 During the lifespan of a cloud service instance it may be required to remain stateful and keep state data cached in memory, even when idle.

State Data to Repository

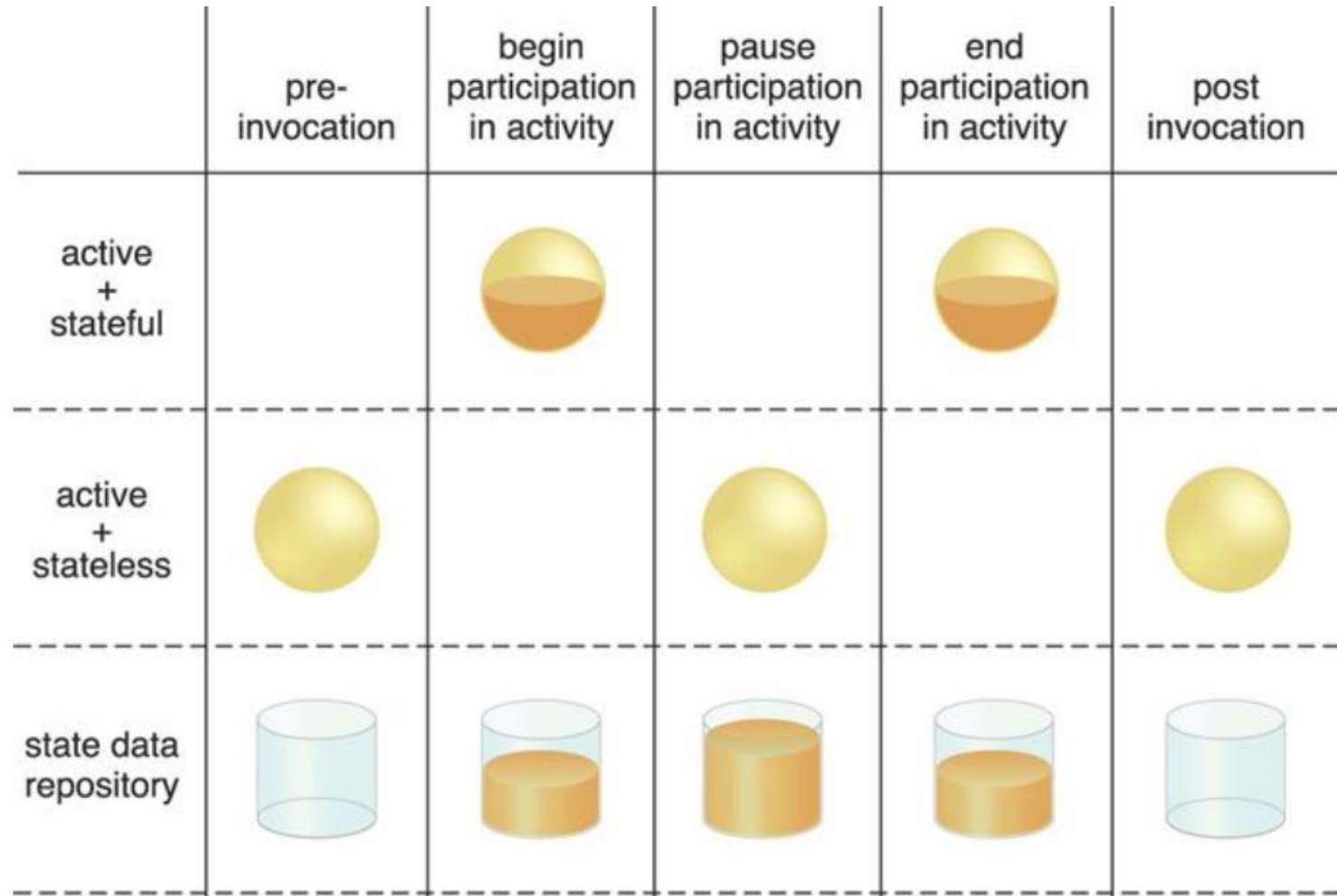


Figure 8.38

By deferring state data to a state repository, the cloud service is able to transition to a stateless condition (or a partially stateless condition), thereby temporarily freeing system resources.

Conclusion

- Automated Scaling Listener
- Load Balancer
- SLA Monitor
- Pay-per-use Monitor
- Audit Monitor
- Failover System
- Hypervisor
- Resource Cluster
- Multi-Device Broker
- State Management Database

Q&A

Any Questions?

